

GutterMOneteringSystem

February 25, 2025

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
↳ Dropout, Input
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img,
↳ img_to_array
import cv2
import os
```

```
[2]: # Step 1: Load and Preprocess the Image Data
data_dir = r'C:\Users\NIKKI\OneDrive\Desktop\Gutter_mon'
```

```
[3]: # Ensure the directory exists and has necessary permissions
if not os.path.exists(data_dir):
    raise ValueError(f"Data directory {data_dir} does not exist.")
if not os.access(data_dir, os.R_OK):
    raise PermissionError(f"Permission denied: Cannot access {data_dir}.")

train_datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

train_generator = train_datagen.flow_from_directory(
    data_dir,
    target_size=(150, 150), # Resize images to 150x150
    batch_size=32,
    class_mode='binary', # Change to 'categorical' for multiple classes
    subset='training'
)

val_generator = train_datagen.flow_from_directory(
    data_dir,
    target_size=(150, 150),
    batch_size=32,
```

```

class_mode='binary',
subset='validation'
)

```

Found 152 images belonging to 2 classes.

Found 38 images belonging to 2 classes.

```

[5]: # Step 2: Build the CNN Model
model = Sequential([
    Input(shape=(150, 150, 3)),
    Conv2D(32, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid') # Change to Dense(num_classes,
    ↪activation='softmax') for multiple classes
])

model.compile(optimizer='adam', loss='binary_crossentropy',
    ↪metrics=['accuracy'])

```

```

[6]: # Step 2: Build the CNN Model
model = Sequential([
    Input(shape=(150, 150, 3)),
    Conv2D(32, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid') # Change to Dense(num_classes,
    ↪activation='softmax') for multiple classes
])

```

```
model.compile(optimizer='adam', loss='binary_crossentropy',  
             metrics=['accuracy'])
```

```
[7]: # Model Summary  
model.summary()
```

Model: "sequential_1"

Layer (type) ↳ Param #	Output Shape	
conv2d_3 (Conv2D) ↳ 896	(None, 148, 148, 32)	↳
max_pooling2d_3 (MaxPooling2D) ↳ 0	(None, 74, 74, 32)	↳
conv2d_4 (Conv2D) ↳ 18,496	(None, 72, 72, 64)	↳
max_pooling2d_4 (MaxPooling2D) ↳ 0	(None, 36, 36, 64)	↳
conv2d_5 (Conv2D) ↳ 73,856	(None, 34, 34, 128)	↳
max_pooling2d_5 (MaxPooling2D) ↳ 0	(None, 17, 17, 128)	↳
flatten_1 (Flatten) ↳ 0	(None, 36992)	↳
dense_2 (Dense) ↳ 4,735,104	(None, 128)	↳
dropout_1 (Dropout) ↳ 0	(None, 128)	↳
dense_3 (Dense) ↳ 129	(None, 1)	↳

Total params: 4,828,481 (18.42 MB)

Trainable params: 4,828,481 (18.42 MB)

Non-trainable params: 0 (0.00 B)

```
[8]: # Step 3: Train the Model
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10, # Adjust based on dataset size
    verbose=1
)
```

C:\Users\NIKKI\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.

```
self._warn_if_super_not_called()
```

Epoch 1/10

5/5 7s 759ms/step -

accuracy: 0.8614 - loss: 0.2472 - val_accuracy: 1.0000 - val_loss: 3.1011e-21

Epoch 2/10

5/5 3s 577ms/step -

accuracy: 1.0000 - loss: 5.9268e-18 - val_accuracy: 1.0000 - val_loss: 0.0000e+00

Epoch 3/10

5/5 3s 579ms/step -

accuracy: 1.0000 - loss: 2.1503e-28 - val_accuracy: 1.0000 - val_loss: 0.0000e+00

Epoch 4/10

5/5 3s 560ms/step -

accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00

Epoch 5/10

5/5 3s 561ms/step -

accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00

Epoch 6/10

5/5 3s 574ms/step -

accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00

Epoch 7/10

5/5 3s 565ms/step -

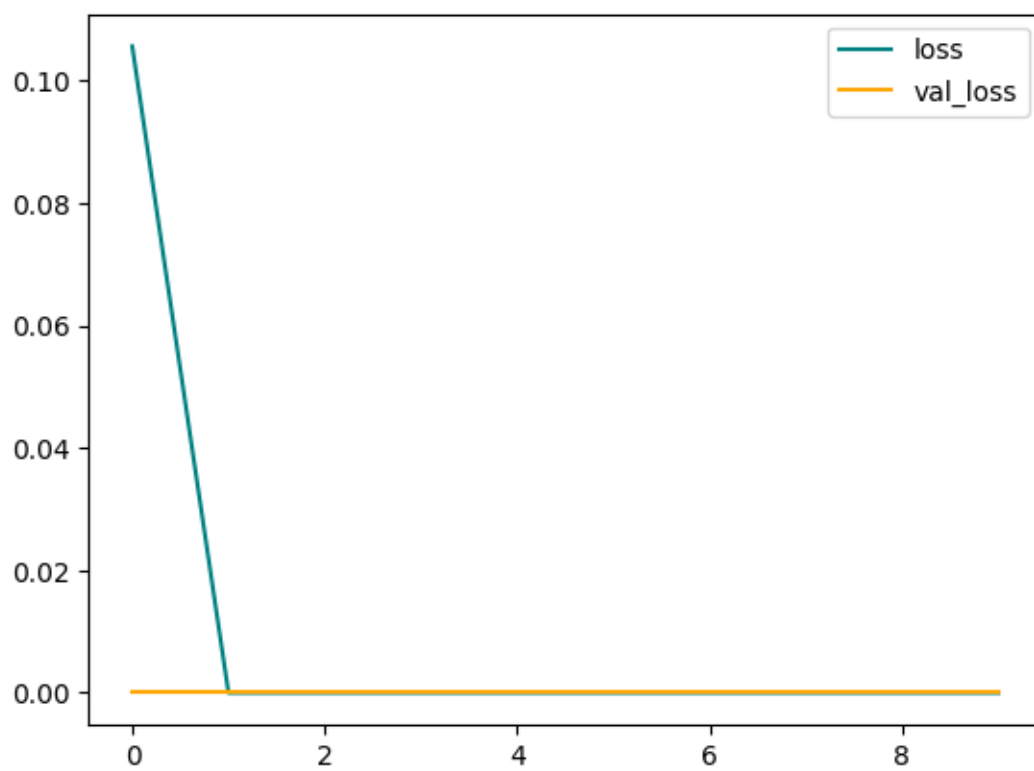
accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00

```
Epoch 8/10
5/5          3s 570ms/step -
accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss:
0.0000e+00
Epoch 9/10
5/5          3s 558ms/step -
accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss:
0.0000e+00
Epoch 10/10
5/5          3s 605ms/step -
accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss:
0.0000e+00
```

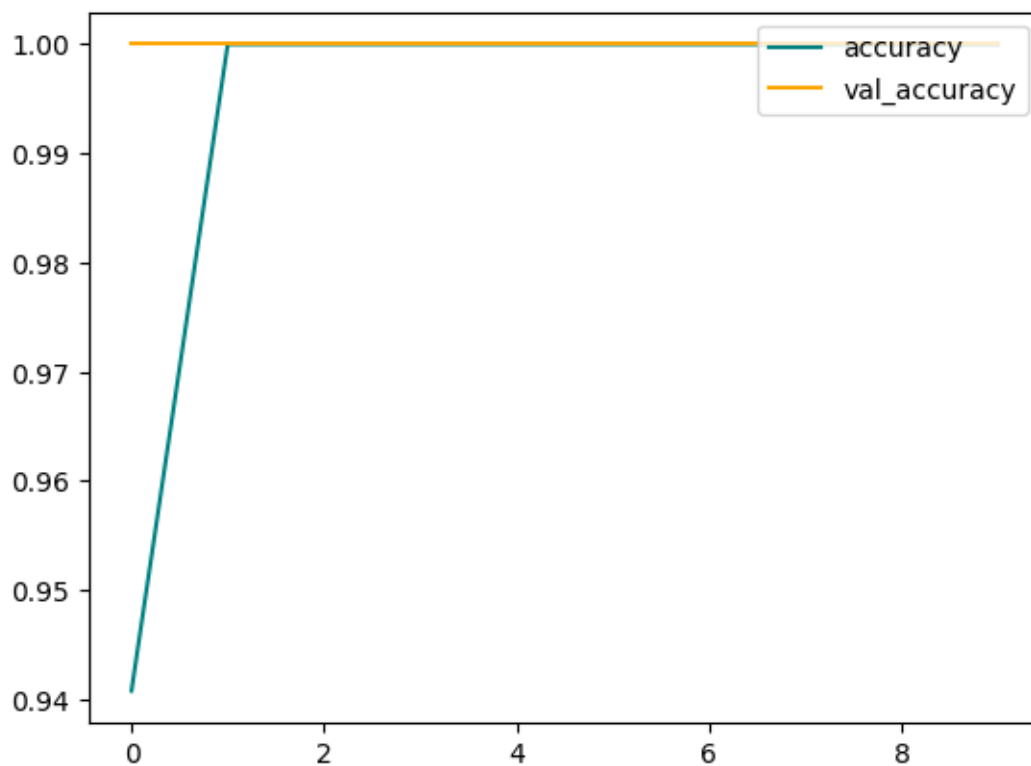
```
[11]: # Step 4: Evaluate the Model
fig = plt.figure()
plt.plot(history.history['loss'], color='teal', label='loss')
plt.plot(history.history['val_loss'], color='orange', label='val_loss')
fig.suptitle('Loss', fontsize=20)
plt.legend(loc="upper right")
plt.show()

fig = plt.figure()
plt.plot(history.history['accuracy'], color='teal', label='accuracy')
plt.plot(history.history['val_accuracy'], color='orange', label='val_accuracy')
fig.suptitle('Accuracy', fontsize=20)
plt.legend(loc="upper right")
plt.show()
```

Loss



Accuracy



```
[12]: # Print Accuracy in Percentage
train_acc = history.history['accuracy'][-1] * 100
val_acc = history.history['val_accuracy'][-1] * 100

print(f"Final Training Accuracy: {train_acc:.2f}%")
print(f"Final Validation Accuracy: {val_acc:.2f}%")
```

Final Training Accuracy: 100.00%
Final Validation Accuracy: 100.00%

```
[33]: # Print Accuracy in Percentage
train_acc = history.history['accuracy'][-1] * 100
val_acc = history.history['val_accuracy'][-1] * 100

print(f"Final Training Accuracy: {train_acc:.2f}%")
print(f"Final Validation Accuracy: {val_acc:.2f}%")
```

Final Training Accuracy: 100.00%
Final Validation Accuracy: 100.00%

```
[14]: # Step 5: Test the Model on New Images
def predict_image(image_path):
    try:
        if not os.path.isfile(image_path):
            return f"Skipping non-file: {image_path}"
        img = load_img(image_path, target_size=(150, 150))
        img_array = img_to_array(img) / 255.0
        img_array = np.expand_dims(img_array, axis=0)

        prediction = model.predict(img_array)
        return "Blocked" if prediction > 0.5 else "Clean"
    except Exception as e:
        return f"Error processing image {image_path}: {str(e)}"
```

```
[15]: # Predict images in Gutter Monitoring folder
image_dir = r'C:\Users\NIKKI\OneDrive\Desktop\Gutter_mon'
```

```
[16]: try:
    for image_name in os.listdir(image_dir):
        image_path = os.path.join(image_dir, image_name)
        print(f"Image: {image_name} -> Prediction: {predict_image(image_path)}")
except PermissionError:
    print("Permission denied: Unable to access some files in the directory.")
```

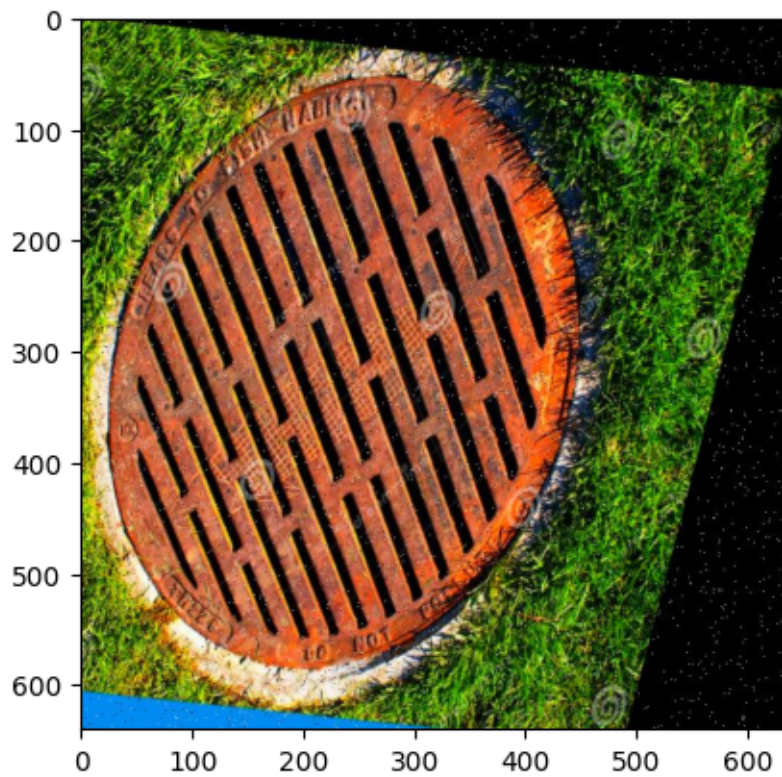
```
Image: images -> Prediction: Skipping non-file:
C:\Users\NIKKI\OneDrive\Desktop\Gutter_mon\images
Image: labels -> Prediction: Skipping non-file:
C:\Users\NIKKI\OneDrive\Desktop\Gutter_mon\labels
```

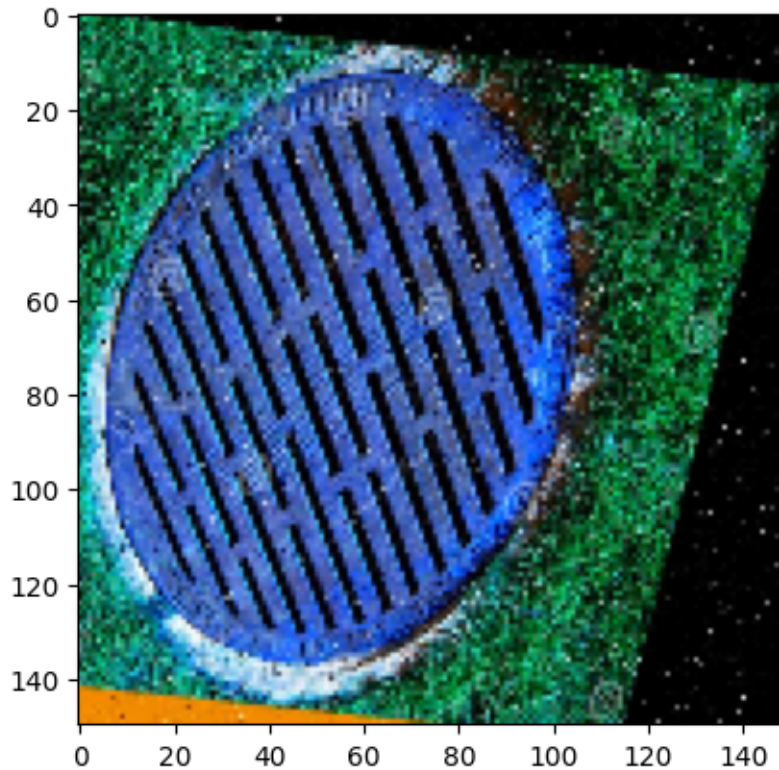
```
[23]: # Step 6: Test an Individual Image
img_path = r'C:\Users\NIKKI\OneDrive\Desktop\Gutter_mon\images\Image_11_jpg.rf.
↳2b5ea7daadc57acbf04b42c463deec70.jpg'
img = cv2.imread(img_path)
if img is None:
    raise ValueError(f"Error: Could not read image {img_path}. Please check the
↳file path.")

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # Ensure correct color
↳representation
plt.show()

resize = tf.image.resize(img, (150, 150))
resize = np.expand_dims(resize.numpy() / 255.0, axis=0) # Normalize and expand
↳dims
plt.imshow(resize[0])
plt.show()
```

```
yhat = model.predict(resize)
print(f'Prediction Confidence: {yhat}')
if yhat > 0.5:
    print(f'Predicted class is Blocked')
else:
    print(f'Predicted class is Clean')
```





1/1 0s 115ms/step
 Prediction Confidence: [[0.]]
 Predicted class is Clean

```
[24]: img_path = r'C:\Users\NIKKI\OneDrive\Desktop\Gutter_mon\images\Image_33_jpg.rf.
      ↪16f5910619ce59312c3c2c38e008e682.jpg'

      # Read image using OpenCV
      img = cv2.imread(img_path)
      if img is None:
          raise ValueError(f"Error: Could not read image {img_path}. Please check the
          ↪file path.")

      # Convert BGR (OpenCV format) to RGB
      img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

      # Resize image correctly
      img_resized = cv2.resize(img, (150, 150)) # OpenCV handles resizing correctly

      # Normalize and reshape for model prediction
      img_array = np.array(img_resized, dtype=np.float32) / 255.0 # Convert to float
      ↪and normalize
```

```

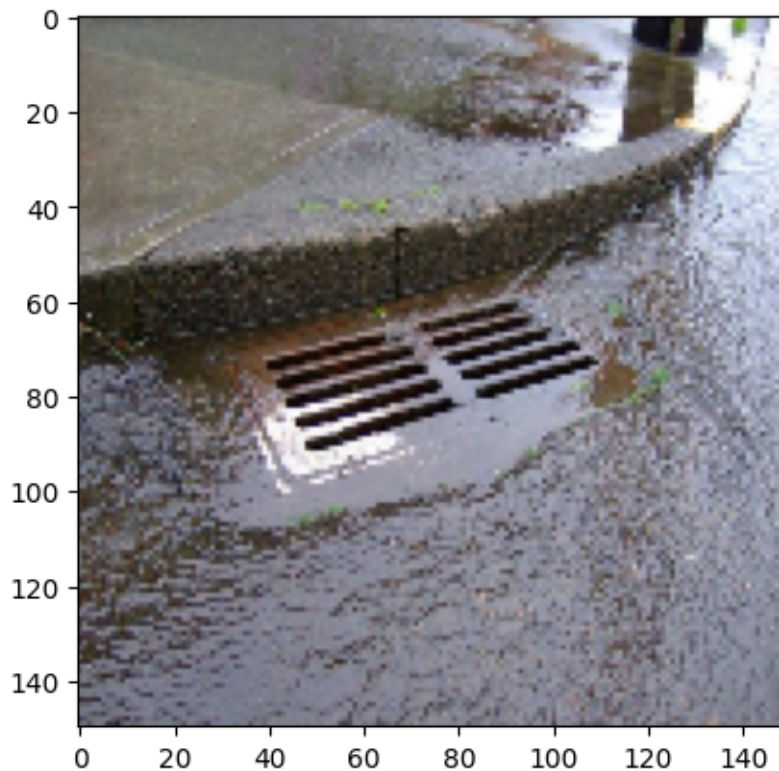
img_array = np.expand_dims(img_array, axis=0) # Add batch dimension

# Display preprocessed image
plt.imshow(img_resized)
plt.show()

# Predict using the model
yhat = model.predict(img_array)
print(f'Prediction Confidence: {yhat[0][0]}')

# Classify based on threshold
if yhat[0][0] > 0.5:
    print(f'Predicted class is Blocked')
else:
    print(f'Predicted class is Clean')

```



```

1/1          0s 105ms/step
Prediction Confidence: 0.0
Predicted class is Clean

```

```

[25]: # Save the trained model
model.save("gutter_monitoring_model.keras")

```

[]: